

AlgoCamp AI Engineering Notes — LLM Fundamentals: From Neural Nets to GPT

1. What an LLM Actually Is

Strip away the hype and an LLM is one thing: **a neural network trained to predict the next token in a sequence.** That's it. Everything else — chat, code, reasoning — emerges from doing that one task extremely well at scale.

input tokens [neural network] probability distribution over the next token
"The cat sat on the" "mat" (0.31), "floor" (0.12), "roof" (0.04), ...

The model picks a next token, appends it, and feeds the whole thing back in to predict the *next* next token — over and over. A “next-token generator” run in a loop is what produces paragraphs, essays, and code.

The mental hook for the cohort: don't think “the model knows things.” Think “the model has learned, from enormous text, what token statistically tends to come next.” Capability is a side effect of very good next-token prediction.

2. The AI ML DL Hierarchy

Nested subsets, from broad to specific:

Artificial Intelligence	machines doing "smart" things
Machine Learning	learn patterns from data (not hand-coded rules)
Deep Learning	neural networks, many layers
LLMs / Transformers	

How humans learn the ML analogy

AlgoCamp anchored it in human learning: we recognize patterns through **experience**. See enough cats, and you can identify a new cat you've never seen. **Machine learning** does the same — feed it many labeled examples, and it learns the pattern well enough to predict on new, unseen data.

- **Traditional programming:** you write the rules. `if temperature > 30 "hot"` .
- **Machine learning:** you show examples; the model *infers* the rules. Show it thousands of (temperature, label) pairs and it learns the boundary itself.

ML Deep Learning

Deep learning is ML using **neural networks** — loosely inspired by how the brain's neurons connect. "Deep" = many layers stacked, letting the model learn increasingly abstract patterns (edges shapes objects, or characters words meaning).

Examples AlgoCamp used to ground it: predicting exam-paper-leak incidents from historical data, or modeling stock-market movements — both are "learn a pattern from history, predict the future" tasks. (Note the honest caveat he raised on stocks: earnings up doesn't cleanly mean price up — real-world signals are noisy and multi-causal. Good reminder that ML models approximate messy reality, they don't capture it perfectly.)

3. Neural Networks as Function Approximators

The core idea

A neural network is a **complex mathematical function** that approximates the pattern hidden in data — especially **non-linear** patterns a straight line can't capture.

Linear vs non-linear (the curve-fitting intuition)

Linear regression: fit the best straight line, minimize error

$y = wx + b$ works when data is roughly linear

Real data is rarely a line:

- •
- • a straight line fits terribly here
- • •
- a neural network fits the CURVE

Linear regression finds the line minimizing the gap between predictions and actual points. But most real relationships bend, branch, and interact. A neural network chains many simple operations with non-linear “activation” functions between them, letting it bend into almost any shape — that’s the “universal function approximator” property.

What “training” means here

Fitting = finding the weights (`w` s) that make the network’s output match the real data as closely as possible. You measure the error (**loss**) and nudge the weights to reduce it, repeatedly. Modern LLMs are this same idea with **billions** of weights and **oceans** of data.

4. Matrix Multiplication & Why GPUs Won

Neural networks are matrix multiplications

Under the hood, running a neural network is mostly **multiplying matrices**. Each layer takes a vector of numbers, multiplies it by a weight matrix, adds a bias, applies a non-linearity, and passes it on. Billions of these multiply-add operations per forward pass.

[input vector] × [weight matrix] [next layer] ...repeated across every layer

CPU vs GPU

	CPU	GPU
Cores	few (4–64), very powerful	thousands, simpler
Strength	complex sequential logic	massive parallel arithmetic
Analogy	a few maths professors	thousands of students each doing one small sum

Matrix multiplication is **embarrassingly parallel** — every cell of the output can be computed independently, at the same time. A GPU's thousands of cores each handle a piece simultaneously; a CPU grinds through them mostly one after another.

The historical point: GPUs were built for graphics (which is also parallel matrix math), but researchers realized the *same* hardware accelerates neural-network training. That repurposing — **before LLMs existed** — is a big part of why deep learning took off when it did. The hardware for parallel matrix math was already there.

5. The Problem LLMs Solved

The pre-LLM pain

Old-school ML: for each task you needed a **specific dataset** and a **separately trained model**. Want spam detection? Collect spam data, train a spam model. Want sentiment? New dataset, new model, new retraining. Expensive, slow, and every new task started from scratch.

What LLMs changed

An LLM is a **general-purpose** model. One model, trained once on broad text, handles translation, summarization, Q&A, code, and more — **without a separate dataset or retraining per task**. You just *ask* it. That generality is the leap: the marginal cost of a new task dropped from “collect data + retrain” to “write a prompt.”

This is the whole premise of *AI engineering* as a discipline — you build on top of these general models rather than training bespoke ones.

6. The Transformer & Attention (the 2017 breakthrough)

“Attention Is All You Need” (2017)

The transformer architecture, introduced in this paper, is the foundation of every modern LLM (GPT, Claude, Llama, etc.).

What it fixed — the RNN/LSTM limitation

Before transformers, sequence models were **RNNs / LSTMs** that read text **one token at a time, in order**, carrying a “memory” forward. Two problems:

1. **Sequential** — can’t parallelize across the sequence, so training is slow.
2. **Forgetful** — long-range dependencies fade; by the end of a long sentence the start is a blur.

The attention mechanism

Attention lets every token look at every other token *directly* and weigh how much each one matters for understanding it — regardless of distance.

"The animal didn't cross the street because it was too tired."

attention lets "it" look back and weight "animal" heavily
the model resolves what "it" refers to

Two wins:

- **Context** — words are understood in relation to *all* other words, capturing meaning that depends on surrounding text.
- **Parallelism** — all tokens are processed together (not one-by-one), which is exactly what GPUs love (§4). This is what made training at massive scale feasible.

7. Tokenization — Turning Text into Numbers

Neural networks do maths on numbers, not letters. **Tokenization** converts raw text into a sequence of integer **token IDs** the model can process.

Tokens aren’t words

A token is often a **sub-word chunk**, not a whole word. Common words are one token; rarer words split into pieces.

"tokenization"	["token", "ization"]	[15496, 1634]
"the"	["the"]	[1169]
"AlgoCamp"	["Algo", "Camp"]	[2348, 6488]

Byte Pair Encoding (BPE)

The dominant algorithm. It starts from individual bytes/characters and **iteratively merges the most frequent adjacent pairs** into new tokens, building a vocabulary that balances “few tokens per text” against “manageable vocabulary size.”

```
start: t h e   c a t
merge frequent pairs repeatedly   "th", "the", "cat" become single tokens
result: a vocabulary where common sequences are single tokens
```

Bits & bytes (the aside)

A byte = 8 bits. BPE literally can begin at the **byte** level (byte-level BPE), which is why it can tokenize *any* text — including emoji, code, and languages it wasn't specifically designed for — nothing is out of vocabulary because worst case it falls back to raw bytes.

8. How Generation Actually Works

Probability distribution sample repeat

The model doesn't “know” the next word — it outputs a **probability distribution over the entire vocabulary**, then a token is chosen and fed back in.

```
"The sky is"   NN   { "blue": 0.62, "clear": 0.11, "falling": 0.02, ... }
                pick "blue"
"The sky is blue" NN { ".": 0.4, "and": 0.15, "today": 0.08, ... }
                pick "."
... loop until done
```

The final layer uses **softmax** to turn raw scores into probabilities that sum to 1:

```
softmax(x_i) = e^(x_i) / Σ e^(x_j)   # bigger scores   bigger probabilities
```

Why the same question gives different answers

A question came up: why does an LLM answer differently to the identical prompt? The precise reason is **sampling randomness**, controlled by **temperature**:

- The model produces the *same* probability distribution each time (given the same input).

- But generation **samples** from that distribution rather than always taking the top token. **Temperature** scales how random that sampling is — higher temperature flattens the distribution (more varied/creative), temperature near 0 makes it near-deterministic (always the top token).

*Clarification worth making to students: the variation comes from **stochastic sampling** / **temperature**, not from models “using different token windows.” Set temperature to 0 (greedy decoding) and a model will give you essentially the same answer every time. This is a useful knob to teach explicitly because learners hit it immediately when calling the API.*

Where does the “knowledge” live? (an important clarification)

A student asked where the tokenized data is *stored*. This is worth getting exactly right, because it’s a common confusion:

- During **training**, the model doesn’t store the text. The patterns get **compressed into the model’s weights (parameters)** — billions of numbers. The training corpus is *not* kept around; its statistical regularities are baked into the weights.
- **Vector databases** are a *different* thing that comes up later, in **retrieval-augmented generation (RAG)**: you store **embeddings** (vector representations of your documents) in a vector DB and fetch relevant ones at query time. That’s an *application pattern built on top of* an LLM — not where the model’s own training knowledge lives.

So: **model knowledge** = **weights**; **RAG document lookup** = **vector DB**. Keeping these separate avoids a lot of downstream confusion when we get to embeddings and RAG.

9. The Three-Phase Training Pipeline

This is the heart of the lecture. A helpful assistant is built in three stages.

Phase 1: PRE-TRAINING	learn language from raw internet text	"base model"
Phase 2: SFT	learn to be a helpful assistant	"instruct model"
Phase 3: RLHF	learn to prefer better answers	"aligned model"

Phase 1 — Pre-training

Train on a **vast** corpus of raw internet text (collected largely via **web crawling**), with one objective: predict the next token. No labels, no curation of “good answers” — just oceans

of text. The model learns grammar, facts, reasoning patterns, code — everything latent in the statistics of human writing.

Output: a **base model** (see §10).

Phase 2 — Supervised Fine-Tuning (SFT)

The base model can continue text but isn't a helpful *assistant*. SFT retrains it on **curated multi-turn conversations** — human-written examples of (user asks, assistant responds *well*). The model learns the *format and behavior* of being helpful.

Training example (curated by humans):

User: How do I reverse a string in Python?

Assistant: You can use slicing: `s[::-1]`. For example...

Companies like **Scale AI** and **Innodata** specialize in producing these labeled datasets — armies of human labelers writing and rating responses.

Phase 3 — RLHF (Reinforcement Learning from Human Feedback)

The final polish. Humans **rank** multiple model responses; those rankings train a **reward model** that scores answer quality. The LLM is then optimized (via RL, classically **PPO**) to produce responses the reward model rates highly.

The mock-test analogy AlgoCamp used: a student takes practice tests (generates answers), a teacher grades them (reward model), and the student adjusts to score better next time. RLHF is that feedback loop, automated.

LLM generates 2+ answers humans rank them train reward model
RL optimizes the LLM to maximize reward-model score better, safer answers

*Precision note for the notes: RLHF-with-PPO was pioneered largely at **OpenAI** and **Anthropic**. **DeepSeek's** notable recent contribution is on the reasoning side — using large-scale RL (their GRPO method, DeepSeek-R1) to elicit reasoning, which is related but distinct from classic RLHF alignment. Worth stating cleanly so students don't merge the two threads.*

10. Base Models — What They Can and Can't Do

What a base model is

The raw output of **pre-training** — e.g. **GPT-2, GPT-3, Llama 3 (base)**. Pure next-token predictors.

Capabilities

- Generate fluent, coherent continuations of text.
- Produce probability-based next tokens that reflect patterns in their training data.

Limitations (why SFT/RLHF matter)

- **Not an assistant** — give a base model a question and it might continue with *more questions* (because that's what the text pattern suggests), rather than answering.
- **Can't reliably do complex tasks** like careful multi-step math out of the box.
- **Can't memorize new information** at inference — its knowledge is frozen in its weights from training.
- **No agentic behavior** — it won't take actions or follow instructions reliably until post-trained.

“Base” vs “foundational” model

A terminology question came up: these terms are **often used interchangeably**.

“Foundation model” is the broader term (any large pre-trained model you build on); “base model” specifically emphasizes the pre-trained, pre-SFT artifact. In casual use they overlap — just know both refer to the pre-trained starting point.

Wide vs deep architecture

Models can grow **wider** (more neurons per layer) or **deeper** (more layers). Both increase parameter count; the balance affects what the model learns and how efficiently. This ties directly into scaling laws (§11) — how you spend your parameter budget matters.

11. Scaling Laws — Compute, Data, Parameters

The central finding

Model performance improves **predictably** as you scale three things together:

More PARAMETERS + more DATA + more COMPUTE predictably lower loss

The remarkable part: the improvement follows **smooth power-law curves** — you can *forecast* how much better a bigger model will be before training it.

The two papers to read (assigned)

1. **“Scaling Laws for Neural Language Models”** — OpenAI (Kaplan et al., 2020).
Established the power-law relationships between loss and scale.
2. **The DeepMind paper** — “Training Compute-Optimal Large Language Models” (Hoffmann et al., 2022), a.k.a. **Chinchilla**. Its key correction: earlier models were **undertrained** — for a given compute budget, you should use **more data** relative to parameters than people had assumed. Chinchilla showed a smaller model trained on more tokens beats a bigger model trained on less.

The practical upshot: it’s not just “make it bigger.” It’s *balancing* parameters and data against your compute budget. This is why “how many tokens should I train on?” became a first-class engineering question.

12. Training Economics

AlgoCamp’s ballpark: training a frontier GPT-class model likely costs on the order of **~\$100 million** — driven by the sheer compute (tens of thousands of GPUs running for months), data pipelines, and the human labeling for SFT/RLHF.

Why this matters for AI engineers: you almost never train these from scratch — the economics are prohibitive. The entire discipline of **AI engineering** is about building valuable products *on top of* models someone else spent nine figures to train, via prompting, RAG, fine-tuning small adapters, and orchestration. Knowing the cost structure explains *why* the field is shaped the way it is.

13. What’s Ahead & Where Traditional ML Fits

- **Next up:** interacting with LLMs through the **OpenAI API** — moving from theory to hands-on.
- **KNN, SVM, and classic ML** — a student asked if these still matter. AlgoCamp’s framing: they’re still useful in the broader ML world, but the **first phase of this cohort focuses on what’s most relevant to AI-engineering jobs today** — i.e.

working with LLMs. Classic algorithms have their place; they're just not the center of gravity for *this* track.

Action Items

AlgoCamp:

- Share the lecture recording within 24h.
- Post both papers (Scaling Laws + the DeepMind/Chinchilla paper) on the dashboard and Discord.
- Upload notes and references to Discord.
- Add the timezone to every lecture notification.

Students:

- Check the announcements for the class schedule.
 - Post lecture questions on Discord.
-

Concept Cheat-Sheet

- **LLM** = a neural network that predicts the next token, run in a loop.
- **AI ML DL LLMs** — nested; ML learns rules from data instead of hand-coding them.
- **Neural network** = a function approximator for non-linear patterns; training = tuning weights to minimize loss.
- **GPUs** win because neural nets are parallel matrix multiplications; thousands of simple cores > few powerful ones.
- **Transformer / attention (2017)** — every token weighs every other token, directly and in parallel; fixed RNN/LSTM's sequential + forgetful limits.
- **Tokenization** — text → sub-word token IDs; **BPE** merges frequent pairs; byte-level BPE handles anything.
- **Generation** — output a probability distribution (softmax), **sample** from it; **temperature** controls randomness (0 = deterministic).
- **Knowledge lives in weights**, not a database. **Vector DBs** are for **RAG** (embeddings retrieval), a separate application layer.
- **3 phases**: pre-training (→ base model) → SFT on curated chats (→ assistant) → RLHF with a reward model + PPO (→ aligned).
- **Base model** = pre-trained only; fluent but not a helpful/agentic assistant; “base” ≈ “foundation” model.

- **Scaling laws** — performance scales predictably with $\text{params} \times \text{data} \times \text{compute}$; **Chinchilla** = train on *more data*, don't just grow params.
- **Training cost** $\approx$ \$100M for frontier models AI engineering = building *on top of* them.

Notes compiled from the AlgoCamp AI Engineering session introducing LLM fundamentals.